

Conceitos sobre API REST (Métodos e HTTP Codes)

Objetivo da Aula

Conhecer os conceitos básicos das APIs REST, no que se refere a tipos de serviços e relacionamento com métodos e códigos do HTTP, e utilizar esse conhecimento para criar um serviço REST simples.

Apresentação

Ao lidar com as APIs REST, devemos sempre lembrar que elas procuram se integrar, de forma natural, ao protocolo HTTP para oferecer seus serviços – que, no caso, envolvem as operações de consulta e gerenciamento de entidades, a partir de rotas específicas. Para que a integração com o ambiente seja efetiva, os serviços REST devem responder da mesma forma que outros servidores HTTP.

Nesse ponto, é necessário conhecer os métodos do HTTP, o que traz melhor compreensão acerca das escolhas arquiteturais do REST, e a especificação de respostas, para que o servidor responda adequadamente às várias ferramentas de depuração e navegação que irão se comunicar com a API.

Conhecendo essas características e o funcionamento básico do REST, será bastante fácil criar um serviço REST, utilizando apenas os elementos do núcleo do NodeJS para oferecer os serviços na web.

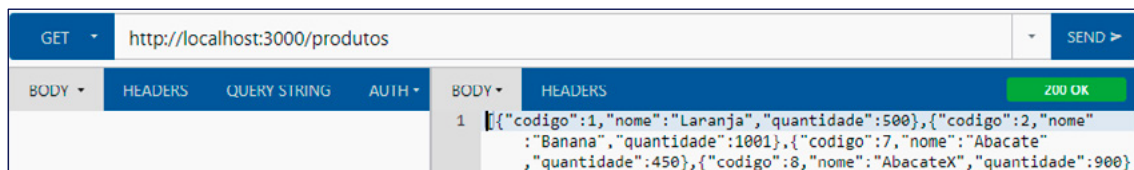
1. Conceitos da Arquitetura REST

A arquitetura **REST**, ou **Representational State Transfer**, utiliza o protocolo HTTP para consultar e gerenciar entidades. Seu nome se deve à forma como a comunicação ocorre, com o uso de JSON ou XML para descrever o valor atual dos atributos de um objeto – ou seja, ocorre, por meio de uma requisição HTTP, a **transferência do estado representacional**.

Por trabalhar diretamente com o HTTP, as entidades são representadas por rotas, de modo que a rota de base, utilizando o método GET, retorna os dados do conjunto completo de entidades, e a rota parametrizada com o identificador, também para o método GET, retorna uma entidade específica.

Um exemplo de utilização pode ser observado na figura seguinte, com uma consulta ao conjunto completo de entidades por meio da rota de base.

Figura 1: Acesso à rota de base via plugin Boomerang



Fonte: Elaboração própria.

Como o REST segue as regras do protocolo HTTP, não temos manutenção de estados, mas o uso de **cache** é viabilizado, de forma direta, permitindo que o tempo para a consulta seja minimizado. Ele também está sujeito ao mesmo modelo de autenticação permitido pelo HTTP, como a autenticação básica, através de informação **no header**, ou o uso de tokens do OAuth.

Devemos interpretar o REST como uma arquitetura cliente-servidor baseada no protocolo HTTP, mas que, diferentemente de um aplicativo web tradicional, utiliza, também, os métodos PUT e DELETE, além do GET e POST. Uma mesma rota pode iniciar ações diferentes, de acordo com o método HTTP utilizado.

Quadro 1: Exemplos de chamadas na arquitetura REST

Método	Rota utilizada	Descrição
GET	/produtos	Retorna os dados de todos os produtos da base de dados no formato JSON.
GET	/produtos/101	Retorna os dados do produto identificado pelo código 101 no formato JSON.
POST	/produtos	Inclui um produto na base de dados. O corpo da requisição terá os valores para inclusão no formato JSON.
PUT	/produtos/102	Altera o produto identificado pelo código 102 na base de dados. O corpo da requisição terá os novos valores no formato JSON.

Método	Rota utilizada	Descrição
DELETE	/produtos/103	Remove o produto identificado pelo código 103 da base de dados.

Fonte: Elaboração própria.

Uma alternativa disponível, mas pouco utilizada pelos desenvolvedores de APIs REST, é a de oferecer código sob demanda. Na prática, isso permite que sejam disponibilizados códigos executáveis a partir do servidor, viabilizando, por exemplo, que seja oferecido um cliente de comunicação com recursos binários, com atualização automática a partir do servidor.

Como a ideia por trás do REST é fornecer um comportamento aderente ao padrão do protocolo HTTP, além de trabalhar com as rotas para a identificação dos recursos e os métodos do HTTP para definição do tipo de operação, a resposta para o cliente deve incluir um status que corresponda aos códigos de resposta que são utilizados no protocolo.

Por exemplo, a consulta deve utilizar um código de resposta **200**, enquanto a inclusão emite o código **201**. Ambos são indicativos de sucesso, mas o código 201 também denota a criação de um recurso no servidor.

2. Métodos do HTTP

Nos sistemas tradicionais para web, apenas os métodos **GET** e **POST** do HTTP são explorados, por meio de formulários HTML e links. Por outro lado, os Web Services **RESTful** exploram outros métodos do protocolo, e o acesso por meio de uma página HTML pode não ser suficiente.



Destaque

Os exemplos apresentados aqui buscam explicar a utilização de Web Services RESTful a partir do cliente, mas necessitam da construção de um serviço adequado para que funcionem.

O conjunto completo de métodos do HTTP pode ser observado a seguir.

Quadro 2: Métodos do protocolo HTTP

Método	Descrição
GET	Tem como objetivo recuperar a informação, a partir da URL, sendo o método mais utilizado.
POST	Envia os dados para o servidor, no corpo da requisição, solicitando a criação de um recurso.
PUT	Envia os dados para o servidor, no corpo da requisição, e um identificador, através da URL, solicitando a substituição do recurso atual.
DELETE	Solicita a remoção de um recurso, no servidor, utilizando a URL para fornecer o identificador desse recurso.
PATCH	Envia os dados para o servidor, no corpo da requisição, e um identificador, através da URL, solicitando a modificação parcial do recurso atual.
HEAD	Semelhante ao GET, mas retorna apenas o cabeçalho da resposta, sem um recurso no corpo.
CONNECT	Permite criar um túnel TCP entre o cliente e o servidor por intermédio de um proxy.
OPTIONS	Retorna as informações de configuração do servidor, sendo utilizado por ferramentas de deploy.
TRACE	Voltado para depuração, faz um teste de loop-back entre o cliente e o servidor.

Fonte: Elaboração própria.

Os métodos GET, POST, PUT e DELETE são utilizados pelo REST, além do PATCH em alguns serviços. Entre esses, o único método que pode ser utilizado diretamente através do navegador é o GET.

Para o método POST, utilizado para inclusão, a partir da rota de base, você poderá utilizar a biblioteca http, como no código da listagem seguinte.

```
const req = http.request(
  {hostname: "localhost", port:3000,
  method: "POST", path:"/produtos"},
  (res) => {
    res.on("data", (d) => console.log(d.toString()));
  }
)
req.write(JSON.stringify({nome:"Abacate", quantidade:450}));
req.end();
```

Inicialmente, é criada uma requisição para **http://localhost:3000/produtos**, em que a URL é configurada através de hostname, port e path, utilizando o método **POST**. Também é

definida uma callback para impressão dos dados recebidos no console, associada ao evento **data**, que será ativado sempre que uma parte da informação enviada pelo servidor for recebida.

Em seguida, o método **write** escreve o conteúdo do corpo, que, no caso, são os dados da entidade que será criada, no formato JSON, em modo texto, seguido da execução da requisição com o método **end**.

Da mesma forma que na inclusão, a alteração pode ser feita com a utilização de um objeto de requisição na listagem seguinte.

```
const req = http.request(  
  {hostname: "localhost", port:3000,  
   method: "PUT", path:"/produtos/1"},  
  (res) => {  
    res.on("data", (d) => console.log(d.toString()));  
  }  
)  
req.write(JSON.stringify({nome:"Abacate",quantidade:390}));  
req.end();
```

O código é semelhante ao da inclusão, mas utiliza o método **PUT** e acessa o endereço **http://localhost:3000/produtos/1**, no qual o último segmento identifica o produto que será alterado. Para o método **DELETE**, bastaria efetuar a troca na configuração da requisição e eliminar a linha com a chamada para **write**, já que o processo de exclusão não envolve o corpo da requisição.

3. Códigos de Resposta do HTTP

As requisições HTTP seguem um padrão de resposta, que é reconhecido por qualquer ferramenta de acesso a URLs, como navegadores, Postman e plugin Boomerang. Uma resposta HTTP completa deve fornecer o tipo de conteúdo, o conteúdo em si e um **código de resposta**.

Com base no código de resposta fornecido, essas ferramentas são capazes de decidir como será a apresentação do conteúdo ou do erro ocorrido. Devido a esse comportamento, nossos Web Services devem fornecer códigos de resposta adequados para cada situação.

A utilização de códigos HTTP corretos também se torna um facilitador para os desenvolvedores que utilizam a sua API, já que esses códigos conseguem detectar, de uma forma simples, a origem dos erros nas requisições que efetuaram.

Os códigos de resposta seguem um padrão de numeração:

- Códigos de **sucesso**: 2xx;
- Códigos de **erro do cliente**: 4xx;
- Códigos de **erro do servidor**: 5xx.

Uma consulta efetuada com sucesso, por exemplo, além do conteúdo, deve fornecer o código **200** na resposta. Já o código **201** seria utilizado para indicar um novo recurso, descrevendo melhor a resposta ao método POST no REST.

Outro código bastante utilizado é o **204**, que serve para indicar um retorno sem conteúdo em uma operação que teve sucesso. Esse seria o melhor código para responder ao método DELETE no REST.

```
const server = http.createServer((req,res)=>{
  const codigo = req.url.substring(req.url.lastIndexOf('/')+1);
  removerProduto(codigo).then(
    () => {
      res.statusCode = 204;
      res.end();
    });
});
// Resposta possível para o método DELETE
```

No código de exemplo para um servidor HTTP, inicialmente, seria obtido o código do produto a ser eliminado no último segmento da URL. Para remover o produto na base de dados, foi invocado o método para exclusão, a partir do controlador, com a passagem do código, sendo retornada a resposta para o cliente ao final do processamento assíncrono.

Observe que a resposta não apresenta conteúdo, mas especifica o código **204**, através do atributo **statusCode**, indicando que a exclusão foi executada com sucesso e a resposta não tem conteúdo para ser apresentado.

Com relação aos erros do cliente ou ao utilizador da API, o código **400** indica uma requisição incorreta, mas é pouco informativo. Utilizando os subgrupos do erro, você pode fornecer informações mais completas para os desenvolvedores que trabalham com sua API.

Em termos de segurança, o código de erro **401** informa que o usuário não foi autenticado, ou seja, não passou por uma validação de senha ou autenticação via OAuth, entre outros modelos. Porém, mesmo um usuário autêntico pode não ter direito de acesso a algum recurso específico, devendo ser utilizado o código **403** para informar que o acesso não foi autorizado (**forbidden**).

O código **404** indica que um recurso não foi encontrado, sendo a origem da famosa mensagem **page not found**. Em termos de uma arquitetura REST, ele pode ser utilizado em uma rota parametrizada, para os métodos GET, DELETE e PUT, quando uma entidade não pôde ser obtida com o identificador fornecido, como no exemplo seguinte, para a consulta.

```
const server = http.createServer((req, res)=>{
  const codigo = req.url.substring(req.url.lastIndexOf('/')+1);
  obterProduto(codigo).then((p) => {
    if(p){
      res.statusCode = 200;
      res.end(JSON.stringify(p));
    } else {
      res.statusCode = 404;
      res.end();
    }
  });
});
// Resposta possível para o método GET
```

Analisando a listagem, você pode observar que ocorre uma tentativa de obter a entidade e, caso o retorno seja um objeto válido, ele é enviado na resposta, junto ao código 200 do

HTTP. Já para a situação em que não temos um objeto, retornamos uma resposta vazia com código 404.

Para indicar que o método HTTP utilizado não é aceito, deve ser utilizado o código **405**. Por exemplo, a rota de base, no REST, é utilizada para consulta e inclusão, através dos métodos GET e POST, e, se a requisição utilizar o método DELETE, deve ser fornecida uma resposta sem conteúdo e com código 405.

O envio de uma resposta com código **429** costuma ser controlado de forma global pelo servidor, pois indica que o usuário efetuou muitas requisições para a mesma URL em um curto período.

Com relação aos erros do servidor, além do código genérico **500**, os mais comuns são: **503**, para informar que o serviço não está disponível; **505**, quando a versão do protocolo HTTP não é suportada; e **511**, indicando que é necessária a autenticação de rede.

4. Criação de Serviço REST Simples

Você pode criar um serviço REST sem a inclusão de módulos de terceiros, utilizando apenas as bibliotecas presentes no núcleo do NodeJS. Temos opções menos trabalhosas, como o express, mas a biblioteca **http** permite tratar todos os métodos do protocolo sem o acréscimo de dependências.



Destaque

Para que os exemplos desta aula funcionem, você precisará de uma entidade Produto, criada com base no Sequelize, e um arquivo controlador, com as funções de gerenciamento do banco de dados, da mesma forma que foi feito em aula anterior.

Considerando que você já tem a entidade Produto e o arquivo para controle das operações de gerenciamento da base de dados, vamos iniciar a codificação pelo método de consulta completo no arquivo **index.js**.

```
import http from "http";
```



```
import {criarProduto, obterTodos, removerProduto, alterarProduto}
    from "../controller-produto.js";
const obterProdutos = (req, res) => {
    obterTodos().then((dados)=>{
        res.statusCode = 200;
        res.end(JSON.stringify(dados));
    });
}
```

A implementação do método é bastante simples, com a execução da consulta pela função `obterTodos` e o subsequente envio dos produtos recuperados para o cliente. Na construção da resposta é utilizado o código **200**, indicando o sucesso da consulta, e o conteúdo é a coleção de produtos transformada em texto.

O encapsulamento da inclusão é um pouco mais complexo, como pode ser observado na listagem seguinte, dando continuidade ao conteúdo de **index.js**.

```
const incluirProduto = (req, res) => {
    let dados = '';
    req.on( 'data', (x) => dados+=x );
    req.on( 'end', () => {
        const produto = JSON.parse(dados);
        criarProduto(produto.nome, produto.quantidade).then(
            (codigo)=>{
                res.statusCode = 201;
                res.end(`Produto criado: ${codigo}`);
            });
    })
}
```

Algo que você deve observar na listagem é o uso dos eventos **data** e **end**, que precisam ser utilizados dessa forma, já que a informação vem fragmentada a partir do cliente. Para o evento **data**, cada parte da informação que chega será concatenada na variável **dados**; já no evento **end**, onde ocorre a sinalização do final de envio, a informação completa é transformada em um objeto no formato JSON, para que seus atributos sejam utilizados na criação do produto.

Após invocar a função `criarProduto`, com a passagem dos valores recebidos, o retorno da função, que é o **código** do novo produto, é enviado na resposta ao cliente, juntamente com o código **201**, informando a criação de um recurso.

Agora, vamos cuidar do processo de exclusão de produto, que utiliza uma rota parametrizada, como pode ser observado a seguir.

```
const excluirProduto = (req, res) => {  
  const codigo = req.url.substring(req.url.lastIndexOf('/')+1);  
  removerProduto(codigo).then(  
    () => {res.statusCode = 204; res.end();});  
}
```

O código do produto é recuperado a partir da rota, no atributo **url** de **req**, com a cópia do valor original a partir da última barra encontrada, na qual temos o uso de **substring** e **lastIndexOf**. Após obter o código, a função `removerProduto` é utilizada, com a passagem do código, e no retorno dela é enviada uma resposta vazia, com código **204**, informando sucesso, apesar da falta de conteúdo.

Finalmente, temos a alteração, conforme pode ser observado a seguir, em que são utilizadas ambas as técnicas das funções anteriores.

```
const modificarProduto = (req, res) => {  
  const codigo = req.url.substring(req.url.lastIndexOf('/')+1);  
  let dados = '';  
  req.on( 'data', (x) => dados+=x );  
  req.on( 'end', () => {  
    const produto = JSON.parse(dados);  
    alterarProduto(codigo,  
      produto.nome, produto.quantidade).then(  
      (dados)=>{  
        res.statusCode = 200;  
        res.end(JSON.stringify(dados));  
      });  
  })  
}
```

O código do produto é recuperado a partir da rota, e os demais valores a partir da informação enviada pelo cliente, no corpo da requisição obtida nos eventos `data` e `end`. Concluída a transmissão da informação, a função `alterarProduto` é chamada, com a utilização do código fornecido na rota e os valores extraídos do corpo da requisição.

Após o processo de alteração ser concluído, seu retorno, que é o produto alterado no formato JSON, é enviado como texto para o cliente, juntamente com o código **200**, informando o sucesso da operação.

Não efetuamos o tratamento de eventuais falhas, mas poderíamos retornar o código **404**, nas funções de alteração e remoção, sempre que o código informado na rota não correspondesse a um produto. Também poderíamos acrescentar a captura de erros, na execução assíncrona, retornando código **500**, na falta de uma opção melhor, para indicar erro interno do servidor.

Todos os tratamentos para as chamadas estão completos, mas ainda falta um servidor que escute a porta correta e direcione as requisições efetuadas para a função de tratamento, de acordo com a rota e o método HTTP utilizado. Agora, vamos completar o código do arquivo **index.js**, com a configuração do servidor, de acordo com a listagem seguinte.

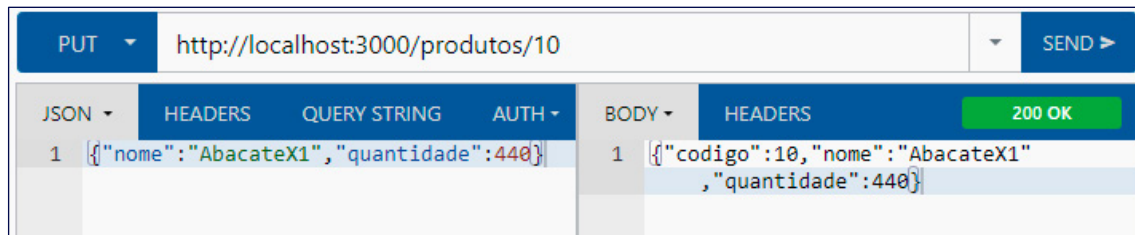
```
const server = http.createServer((req, res)=>{
  if(req.url==="/produtos" && req.method==="GET")
    obterProdutos(req, res);
  else if(req.url==="/produtos" && req.method==="POST")
    incluirProduto(req, res);
  else if(req.url.startsWith("/produtos") &&
    req.method==="DELETE")
    removerProduto(req, res);
  else if(req.url.startsWith("/produtos") &&
    req.method==="PUT")
    modificarProduto(req, res);
  else {
    res.statusCode = 404;
    res.end('404: Not Found');
  }
});
server.listen(3000);
```

Como implementamos todos os tratamentos antes, o servidor terá apenas a função de escutar a rede e direcionar as chamadas para as funções. Quando a URL leva para a **rota de base**, invocamos **obterProdutos** no método **GET** ou **incluirProduto** no método **POST**. Já para uma rota **parametrizada**, detectada com o uso de **startsWith**, invocamos **removerProduto** no método **DELETE** ou **modificarProduto** no método **PUT**.

Temos, ainda, um tratamento que retorna o código **404**, quando não existe uma rota que corresponda à requisição e o servidor começa a escutar a rede ao final, com a chamada para o método **listen**, utilizando a porta **3000**.

Nosso servidor está completo e já pode ser executado, permitindo efetuar as requisições de teste através de uma ferramenta como o Boomerang.

Figura 2: Uso do método PUT no Boomerang



Fonte: Elaboração própria.

Uma opção ao uso de ferramentas como o Boomerang seria a codificação da requisição em JavaScript, utilizando o próprio cliente HTTP do NodeJS.

Considerações Finais da Aula

Aqui, pudemos observar como o REST realmente funciona, como no uso de rotas e métodos específicos do HTTP para cada operação. Também analisamos todos os métodos do protocolo HTTP, para que pudéssemos entender o porquê das escolhas feitas na arquitetura REST.

Em seguida, estudamos os diversos códigos de resposta do HTTP e vimos como trazer uma significância maior para a resposta, algo além da simplicidade do “sucesso” ou “fracasso”. Aprendemos, inclusive, como os diferentes tipos de código são agrupados e classificados.

Com esses elementos em mãos, algum conhecimento do Sequelize e o uso da biblioteca http, pudemos criar um Web Service RESTful simples, mas com respostas que utilizam corretamente os códigos do HTTP, viabilizando que as ferramentas de navegação e testes possam diferenciar, pelo código, o resultado de ações como consulta e inclusão de recurso.

Materiais Complementares



Node.js v20.6.1 documentation

2023, NodeJs.

Esse artigo é parte da documentação oficial da plataforma NodeJS e descreve as funcionalidades do módulo HTTP, com diversos exemplos de utilização.

Link para acesso: <https://nodejs.org/api/http.html> (acesso em 12 set. 2023.)

 Artigo**Códigos de status de respostas HTTP**

2023, Mozilla.

Esse artigo traz uma descrição completa dos códigos de resposta no protocolo HTTP, da instituição Mozilla, que ajuda os desenvolvedores a planejar as respostas de seus serviços, para que estes se integrem melhor com outras ferramentas.

Link para acesso: <https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Status> (acesso em 12 set. 2023.)

Referências

FERREIRA, A. **Interface de programação de aplicações (API) e web services**. São Paulo: Platos Soluções Educacionais, 2021. Disponível em: <https://biblioteca.sophia.com.br/terminal/9564/acervo/detalhe/92072?guid=1691771178058&returnUrl=%2fterminal%2f9564%2fresultado%2flistar%3fguid%3d1691771178058%26quantidadePaginas%3d1%26codigoRegistro%3d92072%2392072&i=5>. Acesso em: 12 set. 2023.

FLANAGAN, D. **JavaScript: o guia definitivo**. Porto Alegre: Bookman, 2014. Disponível em: <https://biblioteca.sophia.com.br/terminal/9564/acervo/detalhe/92164?guid=1691771378051&returnUrl=%2fterminal%2f9564%2fresultado%2flistar%3fguid%3d1691771378051%26quantidadePaginas%3d1%26codigoRegistro%3d92164%2392164&i=8>. Acesso em: 12 set. 2023.

OLIVEIRA, C.; ZANETTI, H. **JavaScript descomplicado: programação para a Web, IoT e dispositivos móveis**. São Paulo: Érica, 2020. Disponível em: <https://biblioteca.sophia.com.br/terminal/9564/acervo/detalhe/92165?guid=1691773379732&returnUrl=%2fterminal%2f9564%2fresultado%2flistar%3fguid%3d1691773379732%26quantidadePaginas%3d1%26codigoRegistro%3d92165%2392165&i=24>. Acesso em: 12 set. 2023.

OLIVEIRA, C.; ZANETTI, H. **Node.js: programe de forma rápida e prática**. São Paulo: Expressa, 2021. Disponível em: <https://biblioteca.sophia.com.br/terminal/9564/acervo/detalhe/92479?guid=1691770593236&returnUrl=%2fterminal%2f9564%2fresultado%2flistar%3fguid%3d1691770593236%26quantidadePaginas%3d1%26codigoRegistro%3d92479%2392479&i=4>. Acesso em: 12 set. 2023.